

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

PRACTICAL - 6

AIM :- To implement a Feed Forward Backpropagation Neural Network learning algorithm to predict whether an employee is likely to receive a promotion based on performance score and years of experience.

Code:- Feed Forward Backpropagation Neural Network

```
#T101 RAJDEEP M PARAB
# AIM: Implement Feed Forward Backpropagation Neural Network
# to predict employee promotion based on performance score and years of experience.
import numpy as np
# Input data:
# [Performance Score, Years of Experience]
X = np.array([[80, 5],
              [60, 2],
              [90, 7]], dtype=float)
# Target output:
# 1 = Promotion
# 0 = No Promotion
y = np.array([1,
              0,
              1]), dtype=float)
# Normalize input data
X = X / np.amax(X, axis=0)
# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
# Derivative of sigmoid
def derivatives_sigmoid(x):
    return x * (1 - x)
# Parameters
epoch = 7000
lr = 0.1
inputlayer_neurons = 2
hiddenlayer_neurons = 3
output_neurons = 1

# Random weights and biases
wh = np.random.uniform(
    size=(inputlayer_neurons, hiddenlayer_neurons)
)
bh = np.random.uniform(
    size=(1, hiddenlayer_neurons)
)
wout = np.random.uniform(
    size=(hiddenlayer_neurons, output_neurons)
)
bout = np.random.uniform(
    size=(1, output_neurons)
```

T101

RAJDEEPP M PARAB

TY.BSc.CS

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

)

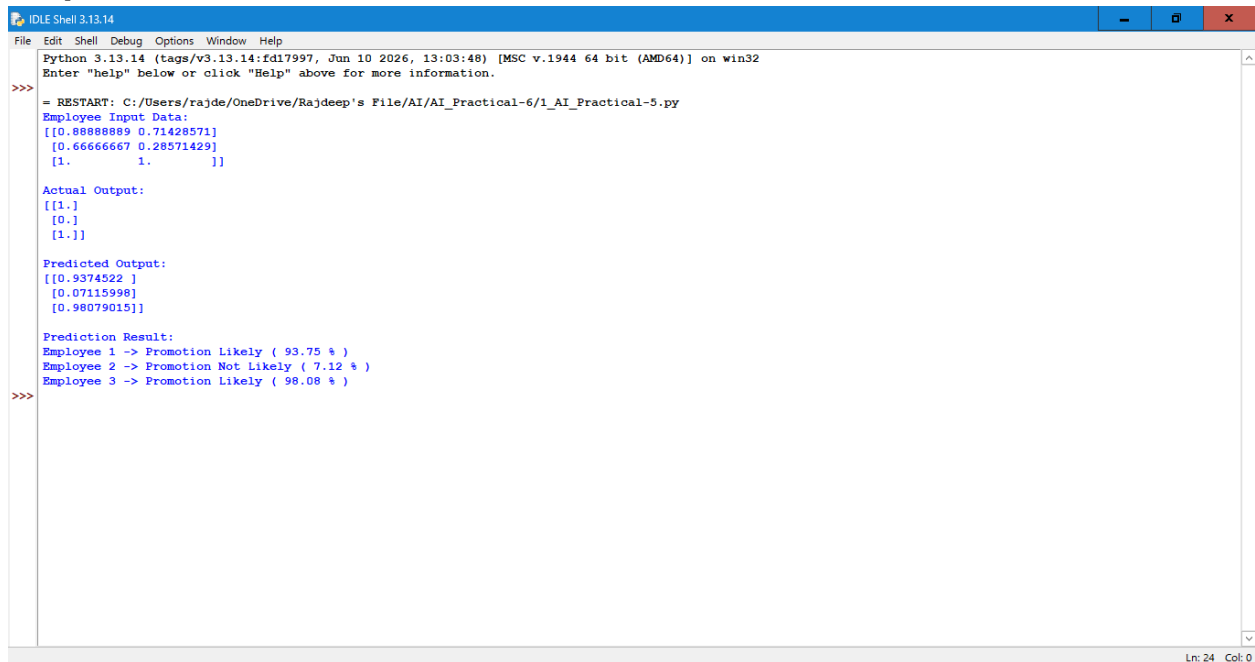
```
# Backpropagation training
for i in range(epoch):
    # Hidden layer
    hinp1 = np.dot(X, wh)
    hinp = hinp1 + bh
    hlayer_act = sigmoid(hinp)
    # Output layer
    outinp1 = np.dot(hlayer_act, wout)
    outinp = outinp1 + bout
    output = sigmoid(outinp)
    # Error
    EO = y - output
    # Output layer gradient
    outgrad = derivatives_sigmoid(output)
    d_output = EO * outgrad
    # Hidden layer error
    EH = d_output.dot(wout.T)
    # Hidden layer gradient
    hiddengrad = derivatives_sigmoid(hlayer_act)
    d_hiddenlayer = EH * hiddengrad
    # Update weights and biases
    wout += hlayer_act.T.dot(d_output) * lr
    bout += np.sum(d_output, axis=0, keepdims=True) * lr
    wh += X.T.dot(d_hiddenlayer) * lr
    bh += np.sum(d_hiddenlayer, axis=0, keepdims=True) * lr
print("Employee Input Data:")
print(X)
print("\nActual Output:")
print(y)
print("\nPredicted Output:")
print(output)

# Easy-to-understand result
print("\nPrediction Result:")
for i in range(len(output)):
    probability = output[i][0] * 100
    if probability >= 50:
        print("Employee", i + 1,
              "-> Promotion Likely",
              "(", round(probability, 2), "% )")
    else:
        print("Employee", i + 1,
              "-> Promotion Not Likely",
              "(", round(probability, 2), "% )")
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

Output:



```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/rajde/OneDrive/Rajdeep's File/AI/AI_Practical-6/1_AI_Practical-5.py
Employee Input Data:
[[0.88888889 0.71428571]
 [0.66666667 0.28571429]
 [1.          1.          ]]

Actual Output:
[[1.]
 [0.]
 [1.]]

Predicted Output:
[[0.9374522 ]
 [0.07115998]
 [0.98079015]]

Prediction Result:
Employee 1 -> Promotion Likely ( 93.75 % )
Employee 2 -> Promotion Not Likely ( 7.12 % )
Employee 3 -> Promotion Likely ( 98.08 % )

>>>
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

Code:- Employee Promotion Probability Calculation

```
#T101 RAJDEEP M PARAB
# Employee Promotion Probability Example
employee_examples = dict(
    e1=dict(Perf='H', Exp='H', Train='Y', Attend='Y', Promo='Y'),
    e2=dict(Perf='L', Exp='L', Train='N', Attend='N', Promo='N'),
    e3=dict(Perf='H', Exp='H', Train='Y', Attend='Y', Promo='Y'),
    e4=dict(Perf='H', Exp='L', Train='Y', Attend='Y', Promo='Y'),
    e5=dict(Perf='L', Exp='H', Train='N', Attend='Y', Promo='N'),
    e6=dict(Perf='H', Exp='H', Train='N', Attend='Y', Promo='Y'),
    e7=dict(Perf='L', Exp='L', Train='Y', Attend='N', Promo='N'),
    e8=dict(Perf='H', Exp='L', Train='Y', Attend='Y', Promo='Y'),
    e9=dict(Perf='L', Exp='H', Train='N', Attend='N', Promo='N'),
    e10=dict(Perf='H', Exp='H', Train='Y', Attend='Y', Promo='Y'),
    e11=dict(Perf='L', Exp='L', Train='N', Attend='Y', Promo='N'),
    e12=dict(Perf='H', Exp='L', Train='N', Attend='Y', Promo='Y')
)
total_emp = 12

# Count occurrences of an attribute value
def tot(attribute, value):
    count = 0
    for key, val in employee_examples.items():
        if val[attribute] == value:
            count += 1
    return count

# Calculate conditional probability
def getProbab(attribute, attribval, value):
    count = 0
    for key, val in employee_examples.items():
        if val[attribute] == attribval and val['Promo'] == value:
            count += 1
    probab = count / tot('Promo', value)
    return probab

def main():
    # Prior probabilities
    PPromoYes = tot('Promo', 'Y') / total_emp
    PPromoNo = tot('Promo', 'N') / total_emp

    # Performance probabilities
    PPerfHigh = tot('Perf', 'H') / total_emp
    PPerfLow = tot('Perf', 'L') / total_emp

    # Experience probabilities
    PExpHigh = tot('Exp', 'H') / total_emp
    PExpLow = tot('Exp', 'L') / total_emp

    # Training probabilities
    PTrainYes = tot('Train', 'Y') / total_emp
```

T101

RAJDEEPP M PARAB

TY.BSc.CS

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

```
PTrainNo = tot('Train', 'N') / total_emp

# Attendance probabilities
PAttendYes = tot('Attend', 'Y') / total_emp
PAttendNo = tot('Attend', 'N') / total_emp

# Performance
print("Probability of Promotion for HIGH Performance:")
print(
    "Promotion Yes:",
    round(
        (getProbab('Perf', 'H', 'Y')
         * PPromoYes / PPerfHigh) * 100,
        2
    ),
    "%"
)
print(
    "Promotion No :",
    round(
        (getProbab('Perf', 'H', 'N')
         * PPromoNo / PPerfHigh) * 100,
        2
    ),
    "%"
)
print("\nProbability of Promotion for LOW Performance:")
print(
    "Promotion Yes:",
    round(
        (getProbab('Perf', 'L', 'Y')
         * PPromoYes / PPerfLow) * 100,
        2
    ),
    "%"
)
print(
    "Promotion No :",
    round(
        (getProbab('Perf', 'L', 'N')
         * PPromoNo / PPerfLow) * 100,
        2
    ),
    "%"
)

# Experience
print("\nProbability of Promotion for HIGH Experience:")
print(
    "Promotion Yes:",
    round(
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

```
(getProbab('Exp', 'H', 'Y')
* PPromoYes / PExpHigh) * 100,
2
),
"%")
)
print(
"Promotion No :",
round(
(getProbab('Exp', 'H', 'N')
* PPromoNo / PExpHigh) * 100,
2
),
"%")
)
print("\nProbability of Promotion for LOW Experience:")
print(
"Promotion Yes:",
round(
(getProbab('Exp', 'L', 'Y')
* PPromoYes / PExpLow) * 100,
2
),
"%")
)
print(
"Promotion No :",
round(
(getProbab('Exp', 'L', 'N')
* PPromoNo / PExpLow) * 100,
2
),
"%")
)

# Training
print("\nProbability of Promotion if Training is Completed:")
print(
"Promotion Yes:",
round(
(getProbab('Train', 'Y', 'Y')
* PPromoYes / PTrainYes) * 100,
2
),
"%")
)
print(
"Promotion No :",
round(
(getProbab('Train', 'Y', 'N')
* PPromoNo / PTrainYes) * 100,
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

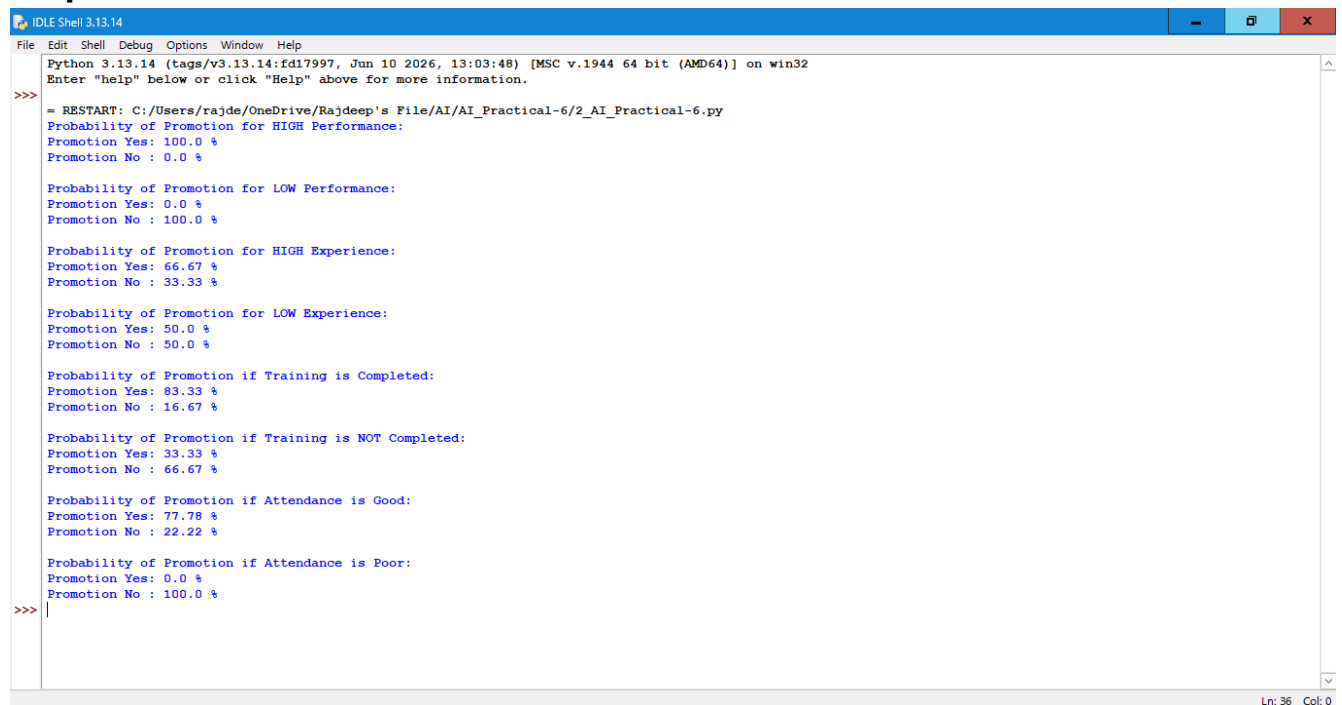
```
2
),
"%%"
)
print("\nProbability of Promotion if Training is NOT Completed:")
print(
    "Promotion Yes:",
    round(
        (getProbab('Train', 'N', 'Y')
         * PPromoYes / PTrainNo) * 100,
        2
    ),
    "%%"
)
print(
    "Promotion No :",
    round(
        (getProbab('Train', 'N', 'N')
         * PPromoNo / PTrainNo) * 100,
        2
    ),
    "%%"
)
# Attendance
print("\nProbability of Promotion if Attendance is Good:")
print(
    "Promotion Yes:",
    round(
        (getProbab('Attend', 'Y', 'Y')
         * PPromoYes / PAttendYes) * 100,
        2
    ),
    "%%"
)
print(
    "Promotion No :",
    round(
        (getProbab('Attend', 'Y', 'N')
         * PPromoNo / PAttendYes) * 100,
        2
    ),
    "%%"
)
print("\nProbability of Promotion if Attendance is Poor:")
print(
    "Promotion Yes:",
    round(
        (getProbab('Attend', 'N', 'Y')
         * PPromoYes / PAttendNo) * 100,
        2
    ),
    "%%"
)
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

```
"%"
)
print(
    "Promotion No :",
    round(
        (getProbab('Attend', 'N', 'N')
         * PPromoNo / PAttendNo) * 100,
        2
    ),
    "%"
)
main()
```

Output:



```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/rajde/OneDrive/Rajdeep's File/AI/AI_Practical-6/2_AI_Practical-6.py
Probability of Promotion for HIGH Performance:
Promotion Yes: 100.0 %
Promotion No : 0.0 %

Probability of Promotion for LOW Performance:
Promotion Yes: 0.0 %
Promotion No : 100.0 %

Probability of Promotion for HIGH Experience:
Promotion Yes: 66.67 %
Promotion No : 33.33 %

Probability of Promotion for LOW Experience:
Promotion Yes: 50.0 %
Promotion No : 50.0 %

Probability of Promotion if Training is Completed:
Promotion Yes: 83.33 %
Promotion No : 16.67 %

Probability of Promotion if Training is NOT Completed:
Promotion Yes: 33.33 %
Promotion No : 66.67 %

Probability of Promotion if Attendance is Good:
Promotion Yes: 77.78 %
Promotion No : 22.22 %

Probability of Promotion if Attendance is Poor:
Promotion Yes: 0.0 %
Promotion No : 100.0 %
>>> |
```

Ln: 36 Col: 0